

To what extent does implementing novel asset placement functions in procedurally generated content, allow assets to be placed without blocking essential pathways?

2204527, Games Academy, Falmouth University

Abstract—This paper aims to explore the effectiveness of using procedural content generation (PCG) to create dungeons for a rogue-like game context, more specifically, automatic asset placement will be implemented into a PCG artifact using three different asset placement functions, referred to as: corridor considered placement, path first placement, and random placement. These algorithms were created for this paper. Their purpose is to create procedurally generated spaces that increase immersion and replayability. This paper aims to analyze whether the results of these asset placement algorithms are good enough to be used for practical application.

Throughout the paper, we discuss challenges that must be overcome in the field of PCG, including the fact that many existing methods only generate basic rooms and corridors and that manually populating these iterations takes a long time. Ethical considerations are also discussed, considering the recent controversy surrounding the use of artificial intelligence, especially in the creative sector. The paper also discusses the creation of a computing artifact to generate and populate dungeons with assets that are then navigated by an agent using A* pathfinding as the method of testing each iteration. The asset placement algorithms are tested and compared to analyze and answer the research question and hypothesis. The aim of this project is that the artifact allows the creation of more engaging and immersive environments, allowing this paper to make a small contribution to the state-of-the-art.

Index Terms—Procedural content generation (PCG), A* pathfinding, rogue-like game context

I. INTRODUCTION

This paper is set in the field of procedural content generation (PCG), PCG is the creation of data using an algorithm, as opposed to manually creating the data [1]. One of the earliest widely accepted cases of using PCG to create random levels was in the game “Rogue” [2], considered to be the progenitor of the rogue-like genre it was released in 1980 and generated its levels each time the player reached the end of the current one using ASCII to represent the environment [3]. There has been a lot of prior work and research done in this field, however during research into the state of the art a gap in the current research was noticed. This is while PCG has been used for the creation of levels often, these usually only necessary content and there was a call for more modular levels by use of in-game assets being imported into generations [4] to make the levels feel more alive. Exploring how this can be done effectively is this paper’s main focus.

To do this, an artifact has been created that contains a PCG method from Sunny Valley Studio [5] which utilizes the random walk [6] and binary space partition [7] algorithms. This method generates procedural dungeons as a series of rooms and corridors. In the case of the corridor considered and random placement algorithms assets are then placed throughout the dungeon by evaluating the available space. The path-finding agent will then try and navigate the dungeon to reach the end, avoiding the placed assets, however for the path first method the dungeon is generated, the path is found and assets are then placed, avoiding the path spaces.

To test the effectiveness of the three asset placement algorithms, each one has been tested an appropriate amount of times according to a sample size generated using GPower [8], using the A* path-finding agent to test if each generation is successful. The end goal of this artifact was that it could reliably generate populated and navigable dungeon levels. This artifact was developed with the Unity game engine [9], which was also used as the main environment in which the content was generated, placed, and tested. The generated content is represented as a dungeon environment inside of Unity as a top-down level. This work has value as assets populating the generated levels has the opportunity to make levels feel more alive thus increasing player immersion and retention as well as saving developers time. If this artifact was modified to fit in a game that can keep players satisfied through a good gameplay loop it would be re-playable which has been shown to improve company loyalty [10].

II. BACKGROUND

A. Challenges

Togelius [11] summarizes several PCG methods and identifies many challenges. These include what form of content generator to use, how to judge whether each iteration is valid etc. There are two main types of PCG content commonly referred to as offline content generation and online content generation. Offline content is created using an algorithm during game development and shipped with the game, it may also be tweaked by a human designer. Online content is created during the runtime of the game, for example, as a player enters a house an algorithm may generate the interior as the player reaches a collider at the door. Each has its

challenges, Togelius states: [11] “Online PCG places two or three main requirements on the algorithm: that it is very fast, has a predictable runtime and that its results are of a predictable quality”. As offline generation is created during game development only so many iterations would be able to be shipped with the game that would likely use more disk space than the PCG algorithm.

One of the challenges that especially smaller studios or indie developers face is that they cannot keep up with the demands of their player bases when it comes to delivering content. While PCG has a variety of uses it can help with this issue by generating more content quickly and cheaply using PCG for things like level generation.

B. A* Path finding

In order to test each iteration, an agent using A* pathfinding was used. An iteration was marked as a success if it could be navigated by the agent using A* pathfinding once the dungeon had been generated and assets had been placed in the case of the corridor considered and random methods. Or if there were no assets on the path tiles in the case of the path first method.

A* pathfinding is a method of pathfinding, the process of finding the shortest route between two points. The agent navigates from the start of the generated level to an exit point on the other side of the level. The A* algorithm has been chosen for this purpose as it is “the most popular algorithm in pathfinding” it has also been heavily documented and tested. [12]

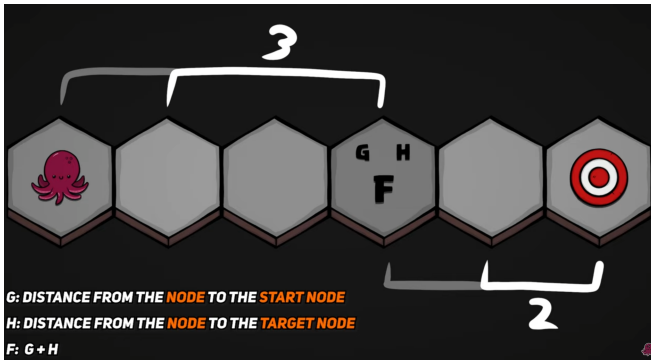


Fig. 1. Screenshot from TaroDev’s video on A* Path finding [13]

In A* pathfinding each movement option is given a score, as seen above these scores are as follows: The G Score; the distance from the start node to the selected node, and the H score; the distance from the node selected to the target node, however it always selects the most optimal route even if this route is not possible. This means the H score will be correct or it will be an underestimate of the length to the target. The F score is a combination of the G and H scores, usually the best path can be found by choosing the movement option that has the lowest F score repeatedly. This information was used to implement A* pathfinding for the agent during testing.

III. CONTEXTUAL ANALYSIS

There has been much investigation into procedural generation. Including PCG for generating dungeons for rogue-like games. As mentioned in the introduction this paper aims to make a small contribution to the state of the art by investigating how importing assets into a dungeon generator may affect its quality and whether the generated levels will be ruined by the assets by making them un-navigable due to having assets block the path to the exit of the dungeon. The artifact uses a placeholder tile (a yellow hexagon) to represent the assets. These are placed throughout the dungeon by the asset-populating functions that evaluate the potential spaces where each asset can be placed. The motivation for this paper is that it may be adapted by solo or small teams of game developers to create levels in a timely fashion that can compete with the levels that can be made by larger teams and studios in a similar time frame.

A. Literature Review

There has been a lot of literature created on PCG. Going back to Togelius’ paper [11] as mentioned this was a broad survey of many different projects and papers involving PCG. Togelius dives into important considerations to be made when working with PCG. One of these is online vs offline content generation discussed in the background. It also discusses necessary vs unnecessary content. Necessary content is content that must be completed by the player to progress in the game like an area that needs to be traversed to retrieve a crucial item. Unnecessary content could be something like a house that the player may or may not enter, which could contain trivial information/items or nothing at all. The key distinction to be made is that necessary content needs to be correct every time otherwise the game will break entirely whereas while unnecessary content should ideally also be correct it is less of a game-breaking problem if it is not. For the computing artifact, the necessary content is the ability of the agent to navigate from the start to the end of the level, while the placement of the assets may seem like unnecessary content at first if they block the path to the exit of the level they will break the game and stop the agent progressing, therefore they are also necessary content. Togelius [11] also highlights the importance of a way to measure PCG algorithms. The artifact for this paper is going to use a simple pass or fail test as mentioned, however some other PCG algorithms such as search-based procedural content generation instead use a different test function that does not just pass or fail the content, but instead grades it with a number value. The test function in this case is called a fitness evaluation, the value assigned to the content is called a fitness score/value. The purpose of this is that more desirable content is assigned a higher fitness score. Iterations that fall below a certain fitness score are discarded while content with a fitness score above a certain value is analyzed to help generate the next set of content to produce content with a higher fitness score.

Thompson, et al. [4] investigate PCG using “Sure footing” [14] a game developed by the authors of the paper. The PCG

algorithm used in this paper is a generate and test-based algorithm. The generate and test method is a heuristic search-based technique based on depth-first search, this being the method for searching a graph or tree-based data structure in which the node at the top of the structure is processed, working through each data path as far as it can before backtracking to find and then explore an unsearched path and repeating this process until all data is processed [15]. Using generate and test means that all solutions are generated and tested to find the best solution.

The evaluation of each iteration created using generate and test is carried out by a heuristic function, this lends itself to PCG creation as it means the most suitable option for the context will be chosen with a high degree of consistency.

Michael, et al. [16] use wave function collapse as well as human input in the design of their PCG algorithm. Wave function collapse (WFC) is a constraint-solving method that can generate tile map levels using several different steps. In this context, a tile is the lowest level element which may be a pixel, image, or level block, and patterns are small rectangular grids of tiles that may be arranged in a 2x2 or 3x3 fashion. When using WFC restraints are required that specify which patterns can be placed next to which other patterns, these may be designed manually or use an example level:

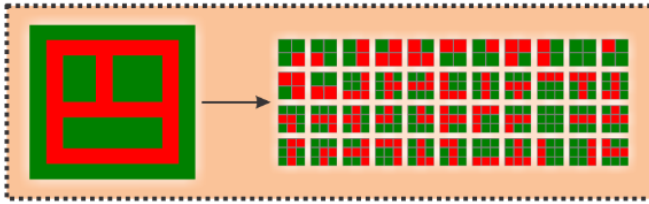


Figure 1: Illustrating the set of patterns (right) that were extracted from an example level (left).

Fig. 2. Example of generated level using set of pre-made patterns and tiles from [16]

First consider the map to be a 2D or 3D matrix of superpositions, super positions being a set of tiles indicating which possible tiles can be placed at a particular place on the map. The map must be initialized as a matrix where each element is a superposition of all the possible tiles, this means that each cell can be filled by any tile.

The next step is to select a tile to collapse and find patterns that are compatible with the current state of the map, for each corresponding tile the superposition in the map contains the tile in the pattern at that location. If there are multiple compatible patterns the selection is randomly selected, weighted by how often each pattern appeared in the example level.

Then in the collapse step, take the current region and replace each cell's superposition with a single tile of the currently chosen pattern, this reduces the number of active superpositions and collapses the region into a concrete pattern.

The last step is propagation, which considers the new state of the map and updates the superpositions of surrounding cells

to take the new information into account, considering all the compatible patterns that may partially overlap with the recently collapsed region. The superpositions of the surrounding tiles are then updated to include only tiles from those patterns. WFC repeats these steps until the level is fully generated.

Xinyu, et al [17] discusses and investigates the use of machine learning in PCG while focusing on the problem of limited training data for specific contexts. The authors discuss machine learning in detail. Machine learning methods create AI based on data, from the perspective of AI training, machine learning methods can be categorized into the following three types: Supervised learning, Unsupervised learning, and Reinforcement learning.

Supervised learning trains AI using input and associated label data, the goal being that when new input data is fed into the constructed AI it produces an appropriate label corresponding to that input. Unsupervised learning involves training AI without the use of labeled data, the only data this method uses is input data, it does not usually produce predictive results and is often used to grasp trends in data. Reinforcement learning trains AI by giving it feedback in the form of a reward. In reinforcement learning the AI is referred to as an agent, it exists in an environment and learns to maximize reward through its actions.

Beukman, et al. [18] observes that a lot of previous research focuses on simple 2D tile map games like Super Mario or Zelda which does not allow the exhibition of more complex structures. It aims to address this by creating a PCG algorithm that decomposes the problem of generating complex levels into smaller manageable pieces.

In this approach multiple generators are hierarchically composed to generate large and complex structures, this is inspired by hierarchical reinforcement learning where a complex task is broken down into smaller components and an agent learns to optimally perform each of those components. An example used by the mentioned paper for their PCG method is breaking down a high-level structure like a town into simple sub-tasks, houses, gardens, etc. Independent agents were trained to generate these individual components.

When factorizing the components of a larger generation task it is easier to make smaller fitness functions for each generator allowing more precise control over them instead of having one giant fitness function for a single more complex generator, breaking the larger task into smaller pieces allows for easier understanding and optimization.

Important points gathered from the literature review are that there is a gap in the current literature for a PCG method that integrates asset placement into generated levels [4], this paper aims to investigate this topic to cover this gap in the existing literature. Also breaking down a larger generation task into smaller functions allows each function to be assessed easier and makes them more manageable. These were considered when going into the development approach as well as when discussing the research question and hypothesis this paper aims to investigate.

IV. DEVELOPMENT APPROACH

A. Computing Artifact Overview

The computing artifact made for this paper has been discussed previously however for further clarity; the artifact consists of three main parts, these being the level generation that uses and adapts the method by Sunny Valley Studio [5], the A* pathfinding method which was adapted from Tarodev's video on the topic [13] as well as the asset populating functions that were created by the author to place assets in the generated levels. The purpose of this artifact is to generate dungeon levels consisting of a series of rooms and corridors that are populated with assets and can be successfully navigated by an agent using A* pathfinding, these are represented in a top-down fashion as shown in Figure 3.

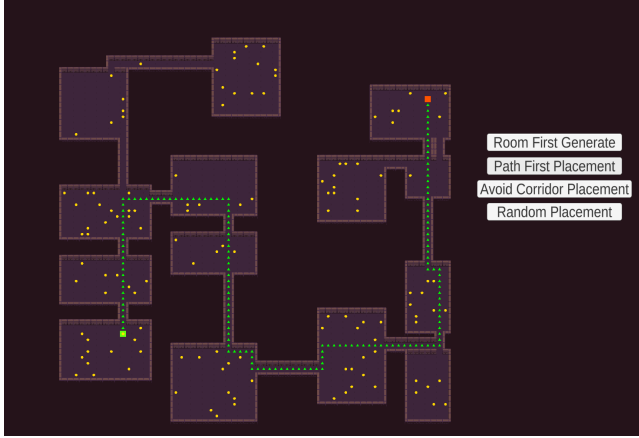


Fig. 3. Screenshot of an iteration of path first asset placement

This screenshot shows the computing artifact in use. The tile set that is used was made by an artist called Pixel Poem [19]. The entrance point of the level is the green square, the exit of the level is the orange square, the assets are represented by the yellow hexagons, and the path that the A* agent found is represented by the green triangles.

The important thing to note is that there is a clear path for the agent to follow that is not blocked by the assets, meaning this example passes the agent's navigation test and is considered a valid iteration. If one of the assets was blocking off a necessary path or there was no route to the end of the level it would be considered an invalid iteration.

The four buttons on the right-hand side of Figure 3 are used for testing each algorithm, the room first generate button runs the generation and pathfinding without any of the asset placement methods while the other 3 buttons run the three asset placement functions respectively.

B. Computing Artifact - Level generation using the binary space partitioning algorithm

For the dungeon generation part of the artifact, a tutorial by Sunny Valley Studios was used [5]. This method makes use of the binary space partitioning algorithm. The binary space partitioning algorithm is a method of recursively splitting up

a space until a necessary requirement is fulfilled [7]. The room and corridors are made by first using the binary space partitioning algorithm to generate the rooms, the corridors are made by linking the centers of these rooms.

C. Computing Artifact - Path finding

For the pathfinding aspect, a method by TaroDev was used [13] and adapted to suit the purposes of the artifact. Each tile is assigned a script called 'SCR_PossiblePathNode', every time the dungeon is generated the positions of each tile are found, and a ray is cast towards each to get its node base component, a script called 'SCR_NodeBase'. These scripts are used to find the neighbors of each tile which are necessary for the pathfinding code.

The pathfinding script 'SCR_Pathfinding' works by creating two lists of node bases, these being 'toSearch' and 'processed', the start node is added to toSearch at the start of generation, it then iterates through this nodes neighbors analyzing the nodes with the lowest F score or H score if the F scores are the same. It picks the node with the lowest score and makes this the new current, searching through the neighbors of that node, repeating this cycle until either a path is found at which it retraces the path and sets each node to be green resulting in a valid iteration or if every available tile has been searched and no path has been found it returns null and is an invalid iteration.

D. Computing Artifact - Asset placement functions

After generation, each iteration's generated rooms and corridors are then evaluated by the 3 different asset placement functions. Each one works slightly differently.

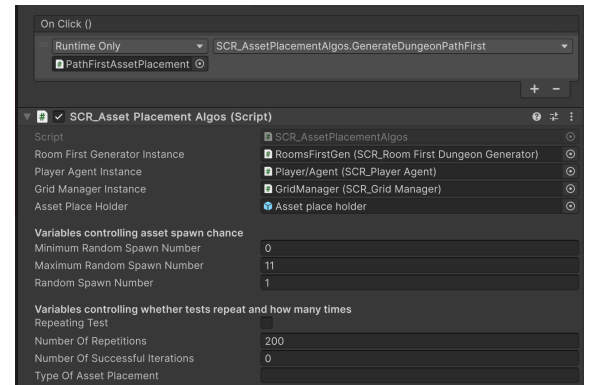


Fig. 4. Screenshot of asset placement script that each generation button uses

Each button uses the script 'SCR_AssetPlacementAlgos', this script contains the methods that each asset placement algorithm uses to generate the dungeon, path, and place assets. As seen in Figure 4 this script has a series of variables, the instance scripts are used to store scripts from objects in the scene that are used in generation and placement. The asset placeholder is the yellow hexagon prefab seen throughout the generation in Figure 3.

The variables controlling the asset spawn chance in this screenshot and also the values used when gathering data are

0 to 11, when deciding whether to place an asset on a tile if it is marked as a possible location for an asset random is used to generate a number between these two values, if it is less or equal to the random spawn number variable an asset is placed. These are made public so they can be altered, raising the random spawn number variable would make the assets more frequently placed throughout the level for example.

The other variables on the script were used for gathering data, these are used to decide whether the test should repeat and how many times and it also keeps track of the number of successful iterations and the type of asset placement that was generating these iterations. These variables were made public to aid in data gathering.

The computing artifact as well as all code for this project can be found on the GitHub repository: <https://github.falmouth.ac.uk/GA-Undergrad-Student-Work-24-25/COMP302-Artifact-Unity>

1) *Path First Asset placement::* As seen in the code snippet labeled listing 1 this placement method works by first setting the type of asset placement being used and then checking whether the generations should repeat or not based on the result of the 'repeatingTest' variable. If it is repeating it starts a co-routine that repeatedly runs a code block for the 'numberOfRepetitions' value.

For this placement method the code consists of generating the dungeon and path, a slight delay to make sure this is done, and then it runs a method called 'CheckTilesNotInPath' this goes through a for loop for every floor tile, if a tile is not in the path tile list that is made when the path is generated it has a chance to spawn an asset on that tiles position, this chance is determined by the random variables that were discussed previously in the overview of all the asset placement functions.

Listing 1. Path first placement code

```
public void GenerateDungeonPathFirst()
{
    Debug.Log("Generating dungeon");
    typeOfAssetPlacement = "Path first asset
    ↳ placement";
    //Generate dungeon and path
    if(repeatingTest)
    {
        StartCoroutine(PathFirstGenRepeating());
    }
    else
    {
        roomFirstGen.GenerateDungeon(true);
        Invoke(nameof(CheckTilesNotInPath),
        ↳ 1.5f);
    }
}

private IEnumerator PathFirstGenRepeating()
{
    for (int i = 0; i < numberOfRepetitions;
    ↳ i++)
    {
        //Resets number of succesful
        ↳ iterations
        if (i == 0)
        {
```

```
        numberOfSuccessfulIterations = 0;
    }
    //Tracks what iteration it is on
    Debug.Log("Running path first
    ↳ placement, generation number: "
    ↳ + (i + 1));
    //generates dungeon and path
    roomFirstGen.GenerateDungeon(true);
    //waits half a second to make sure it
    ↳ does not overlap with dungeon
    ↳ being generated
    yield return new WaitForSeconds(0.5f);
    CheckTilesNotInPath();
    //waits so there is a chance to see
    ↳ the assets placed before moving
    ↳ on to next iteration
    yield return new
    ↳ WaitForSeconds(0.25f);
}
Debug.Log($"Path first placement has
    ↳ succeeded
    ↳ {numberOfSuccessfulIterations}
    ↳ times out of {numberOfRepetitions}
    ↳ iterations!");
}
```

2) *Corridor considered Asset placement::* This placement method works similarly to the previous one discussed, however instead of generating the path before the assets are placed, the assets are placed before the path is generated. It works by again checking if the code is repeating, if it is, it runs the co-routine 'CorridorConsideredGenerationRepeating' for the number of times 'numberOfRepetitions' is set to as seen in listing 2.

This code generates the dungeon, waits to ensure this is done, and then runs 'CheckTilesNotInCorridor' this loops through every tile position in the list of corridor spaces that are made when the dungeon is generated and adds these to the 'corridorTiles' list which is a list of the node bases from each tile which is then used to check if each floor tile is not in that new 'corridorTiles' list, if it is not it has a chance for a tile to spawn on that location. After a delay, the path is then generated.

Listing 2. Corridor considered placement code

```
private IEnumerator
    ↳ CorridorConsideredGenerationRepeating()
{
    for (int i = 0; i < numberOfRepetitions;
    ↳ i++)
    {
        if (i == 0)
        {
            numberOfSuccessfulIterations = 0;
        }
        roomFirstGen.GenerateDungeon(false);
        yield return new
        ↳ WaitForSeconds(0.25f);
        CheckTilesNotInCorridor();
        yield return new
        ↳ WaitForSeconds(0.25f);
        GeneratePath();
        yield return new
        ↳ WaitForSeconds(0.25f);
    }
```

```

    }
    Debug.Log($"Corridor considered placement
    ↳ has succeeded
    ↳ {numberOfSuccessfulIterations}
    ↳ times out of {numberOfRepetitions}
    ↳ iterations!");
}

private void CheckTilesNotInCorridor()
{
    foreach (var tilePosition in
        ↳ roomFirstGen.corridors)
    {
        corridorTiles.Add(
            gridManagerInst.GetTileAtPos(
                tilePosition + new Vector2(0.5f,
                    ↳ 0.5f), true));
    }

    foreach (GameObject floorTile in
        ↳ gridManagerInst.floorTileObjs)
    {
        if (!corridorTiles.Contains(
            floorTile.GetComponent<SCR_NodeBase>()))
        {
            RandomSpawnChance(floorTile,
                ↳ true);
        }
    }
}

```

3) *Random Asset placement*: This placement method works by repeating the steps the previous methods do, checking if the code should repeat, and repeating for the number of times that have been set, it then generates the dungeon, waits, and places assets randomly by iterating through each tile and calling the code to spawn a tile using the random spawn chance code. And then generating the path after.

Something to note for this placement method is that the method that is used to spawn the asset based on chance, 'RandomSpawnChance' is set to mark the tile as not walkable if the 'markAssetPlaced' bool is true which it is for this placement method, this code marks the node base attached to the tile the asset is placed on as having an asset on it, the pathfinding code checks for this when looking through available nodes, so if a tile that has an asset placed on it is in an essential corridor or in the way which blocks the route to the exit the pathfinding and iteration will fail.

Listing 3. Random placement placement code

```

private IEnumerator RandomPlacementRepeating()
{
    for (int i = 0; i < numberOfRepetitions;
        ↳ i++)
    {
        if (i == 0)
            ↳ {numberOfSuccessfulIterations =
            ↳ 0;}
        roomFirstGen.GenerateDungeon(false);
        yield return new WaitForSeconds(0.5f);
        PlaceAssetsRandomly();
        yield return new
            ↳ WaitForSeconds(0.25f);
        GeneratePath();
    }
}

```

```

        yield return new
            ↳ WaitForSeconds(0.25f);
    }
}

private void PlaceAssetsRandomly()
{
    foreach (GameObject floorTile in
        ↳ gridManagerInst.floorTileObjs)
    {
        RandomSpawnChance(floorTile, true);
    }
}

private void RandomSpawnChance(GameObject
    ↳ floorTile, bool markAssetPlaced = false)
{
    var randomNumber =
        ↳ Random.Range(minimumRandomSpawnNumber,
        ↳ maximumRandomSpawnNumber);
    if (randomNumber <= randomSpawnNumber &&
        ↳ !CheckTileNotDoor(floorTile))
    {
        SpawnAsset(floorTile);
        if (markAssetPlaced)
        {
            markTileNotWalkable(floorTile);
        }
    }
}

```

E. Development Method

For the development of the artifact for this paper, the rapid application development cycle of software development was used, this sped up development and allowed quick testing and fixing of any issues which helped make the artifact as promptly as possible. For the case of the artifact that was made for this paper, the benefits outweighed the cons of having a less specific plan as rapid prototyping and testing allowed the identification and solving of issues earlier in the development life cycle leading to a more polished artifact.

The rapid application development cycle (RAD) is based on rapid prototyping, quick feedback, and testing with less emphasis on strict planning [20]. This development method prioritizes development and building a prototype rather than planning. It shines at making multiple iterations and updates to software, without starting from scratch. "This helps ensure that the final outcome is more quality-focused and aligns with the end users' requirements" [20].

F. Quality Assurance

A definition of quality by Dr. W. Edwards Deming states: "Good quality means a predictable degree of uniformity and dependability with a standard suited to the customer" [21]. This quote heavily influenced the definition of quality this paper used. This being that something, in this paper's case a piece of software, is consistently suitable for its intended purpose and can be relied on by the things or people that need it to perform its duty without concern.

As this paper was situated in the games industry the "customer" in Deming's quote was considered to be the player(s)

of a game that contains the artifact produced by this paper. Therefore while producing the artifact there was a strive to produce something that is of a level of quality that would satisfy a player, it was also tested for functionality to ensure there were no major bugs with a goal of no minor bugs either.

To ensure this the artifact was tested rigorously as it was developed. The tests that were used included unit testing, as well as functional and non-functional testing.

Unit tests were developed to ensure the continued functionality of code, this worked well with the artifacts development cycle, RAD, as there was continuous work being added and tested very quickly. Unit tests ensured that different parts of the artifact did not break when adding new functionality which sped along development and prevented blockers of progress. A screenshot of the unit tests being ran can be found in the appendix, the code for these unit tests can be found in the GitHub repository.

Another method of quality assurance used was functional testing and non-functional testing [22], functional testing was done regularly throughout development, paired with the RAD development cycle, this allowed early prototypes to be made which allowed testing to identify shortcomings and pitfalls so they could be remedied early in production. Nonfunctional testing was also used to test the quality of the artifact besides the minimum requirements such as the ease of use.

Functional and nonfunctional testing was done constantly while working on and testing the artifact. The aim was that keeping on top of these quality assurances would produce an artifact of quality as defined at the beginning of this section.

V. SCIENTIFIC QUESTION

The question that this paper seeks to answer is the same as the title of this paper: “To what extent does implementing novel asset placement functions in procedurally generated content, allow assets to be placed without blocking essential pathways?”

This paper aims to answer this question by using the computing artifact that has been discussed. This was tested thoroughly by generating and testing a sample of iterations.

The answer to this question is based on whether the asset populating functions allowed assets to be placed throughout the generated levels while not blocking the path to the exit. If the functions worked properly and generated populated levels that did not block the exit then the question can be answered positively. However, if the assets were not able to be placed effectively and they did block the agent from finding a valid path in each iteration then the answer to the proposed question could be a no, or the artifact may have not been suitable or satisfactory for sufficiently answering the question.

VI. HYPOTHESIS

This paper aimed to test different asset placement methods to determine which would be effective and generate the most valid iterations, this paper’s hypothesis is that among the methods tested, placing assets on floor nodes that are not in the path list would be the most effective method and result

in the least path obstructions, this hypothesis was chosen as the author believes that the other asset placement functions would have issues such as the corridor placement algorithm not detecting corridors properly or the random placement algorithm placing assets in the way of essential paths. This was investigated by implementing the three different asset placement functions: path first placement, corridor considered placement, and random placement. These algorithms were developed and tested an appropriate amount of times as determined by GPower [8] to investigate which generated the most valid iterations and was the most fit for its purpose. This relates back to the research question of determining which asset placement algorithm generates the most valid iterations, which may enable further investigation into the asset placement algorithm of that type and support the idea that automatic asset placement functions can work well with procedurally generated content.

To summarize the hypotheses are:

- Alternative hypothesis: Among the methods tested, the path first placement method will result in the most valid iterations.
- Null hypothesis: There is no significant difference between the number of valid iterations generated by the path first placement method compared to the other two placement methods.

VII. RESEARCH METHODS

A. Philosophical Position

To understand the research and goal of this paper it would be of use to understand the author’s philosophical ontological, epistemological, and axiological positions/opinions [23]. The author’s epistemological position is that we can only state things as fact if they can be witnessed or proved without a shadow of a doubt, however, even things that have not been directly witnessed or proven should still receive attention as they may provide insight or clues that can help get to the truth of the context if there is one that exists or one that we can discover. This position inspired the creation of the artifact this paper uses as well as the question that it proposes.

The author’s ontological position is that humans are best understood on an individual level. The author also believes that whatever is experienced can be considered something that exists in some form and that not everything has to have some sort of higher meaning. The artifact that is going to be created for this paper will serve to gather data to answer the question that the paper proposes.

The author’s axiological position is that good and bad can be categorized by the effect actions and words have on other people and the world as a whole. The author also believes that the paper’s purpose is for good, as while many people dislike the idea of AI-generated content, the artifact generated for this paper, can still be a tool that can work with and aid people instead of harming them by taking away their livelihoods the reasons for this is expanded upon in the ethical considerations section.

Due to these opinions, this paper adopts a positivist philosophical stance, the author believes that the research question and hypothesis provided can be definitively answered if the artifact is of the quality that this paper has endeavored to achieve. This paper aims to answer the proposed question and hypothesis with the data gathered from the artifact with the aspiration that the answers can make a small contribution to the state of the art.

B. Research Approach

This paper takes a mainly quantitative approach to research [24], this is because answering the proposed question requires evidence in the form of data. The data required is based on a sample of iterations generated by the computing artifact. These iterations were assessed using the navigation agent to test whether each iteration can be successfully navigated. If they can be navigated the iteration will be marked as a success, if not then it will be marked as a failure.

Using this method, data was gathered to check how many of the generated iterations can be considered valid, this has been used to answer the proposed question in an objective manner in line with the quantitative approach that this paper aims to take. Quantitative data was also used to inform the results of the hypothesis, more will be covered in the data analysis and results section.

VIII. ETHICAL, PROFESSIONAL, SOCIAL, AND LEGAL CONSIDERATIONS

There has been much discourse on the topic of Artificial Intelligence and its role in the games and art industries. This is one of the main ethical and social concerns of this paper, whether it is morally acceptable to use AI for the generation of content that could be made by humans.

While discussion about this topic in the broader spectrum of the games, art, and other affected industries is outside of this paper's scope. The author believes the artifact created for this paper as well as the question that it aims to answer is ethically and socially acceptable as it is small in scope and aims to be a tool that can be adapted by smaller studios. it also uses assets with appropriate licensing and copyright so it cannot be considered stealing.

For professional considerations, the author believes that content generation can still be outperformed by human designers, also the use of PCG in level design usually only works well with games of certain genres like rogue-likes. For classic single-player games like "Half-Life 2" [25] the use of PCG and AI-generated content will likely always be less effective than using experienced human designers to handcraft a game's levels from scratch, as this will be more detailed and have more thought put into it than PCG currently has the capacity for. On the contrary rogue-like games make good use of PCG as their core gameplay loop is often based on experiencing the same content over and over again while using PCG to change the environment and challenges that the player faces to make it feel fresh and avoid player boredom each time. Therefore

the author believes that the artifact in this paper does not have the potential to negatively affect people's livelihoods.

For legal considerations, these have been handled by using appropriate licensing and credit/referencing throughout the entire development and write-up process. This has been handled by citing the sources that have been studied when discussing the development of the artifact such as Sunny Valley Studio's PCG method [5] and TaroDev's video on A* pathfinding [13]. Also whenever code has been used that has been gathered from online this has been sourced in a citation section at the top of each script.

This paper has been deemed low ethical risk by Falmouth University, the ethics application made for this project can be found here: <https://learningspace.falmouth.ac.uk/mod/forum/discuss.php?d=56639>

IX. DATA GATHERING AND TESTING

For testing the hypothesis to determine whether there was a difference between the asset placement functions performance a chi-square test of independence was used [26]. This test determines whether there is a difference between two variables in a series of data groups, in the case of this paper this was used to test if there was a difference between the rate of passes and fails between each of the asset placement methods.

To gather enough data to compare the asset placement algorithms each one was tested 200 times. Videos of each algorithm being tested can be seen in the results folder inside the GitHub repository, this number was calculated using GPower [8] by inputting the values seen in Figure 5 the effect size, error prob, and power were left as standard. However the degrees of freedom value (df) was calculated to be 2 as this value is determined by multiplying the number of groups to test minus 1 by the number of variables to test minus 1, there were 3 asset placement functions to test and 2 values to test, pass or fail, minus 1 from each and multiplying these values gives 2. These values gave a minimum sample size of 172, it was tested 200 times to give extra coverage and make sure the data represented the results of each algorithm correctly. This data was analyzed using a chi-square test of independence done in Python, code for this can be seen in the appendix and also in the results folder of the repository.

For further insight into how algorithms performed compared to each other in order to determine if the hypothesis is correct post-hoc tests were also performed on the data, these being Fisher exact tests. These were chosen because they provide an exact p-value that clearly represents the difference in variables between the two groups [27].

These tests were used to calculate the difference in passes and fails, comparing each algorithm to the other one at a time until each had been compared to each, this code can also be seen in the appendix. These tests provided p values that represent how differently the algorithms performed compared to each other.

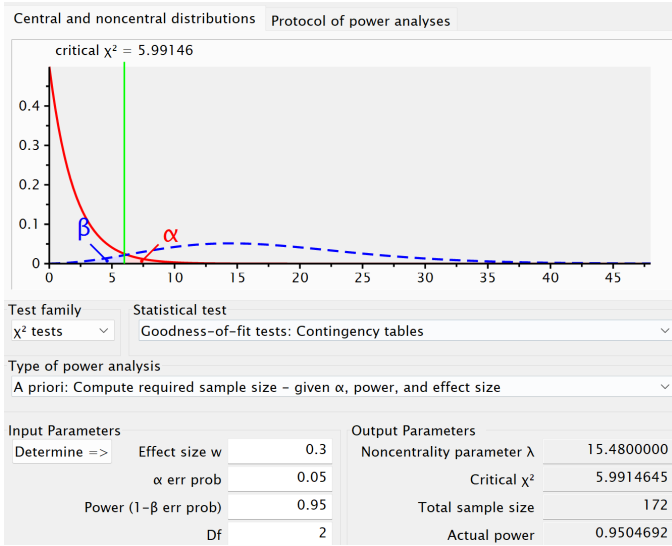


Fig. 5. Samples needed to test each asset placement function according to G*Power [8]

X. RESULTS AND ANALYSIS

After running the chi-square test code the results generated were:

Variables	Values
Test statistic	595.5112
P-Value	0.0000

Table showing the results of the chi-square test

The test statistic represents the statistical difference between the actual data and the data that would be expected if all of the algorithms generated the same data, and therefore performed the same. 595 is a very high value and shows that there is a very clear difference in how at least one of the algorithms performed compared to the others. The p-value represents the chance that the results were random, a value of 0.0000 shows that there is almost no chance that the results were random, giving further credit that the results are correct and not a coincidence.

As seen in Figure 6, the corridor considered placement succeeded 200 times, path first placement succeeded 200 times and random placement succeeded 1 time. These results clearly support the null hypothesis and refute the alternative hypothesis, as both path first and corridor considered placement had the same success rates instead of the path first placement method performing the best out of the three as the hypothesis predicted. However this also answers the research question as two of the algorithms succeeded in every iteration of generation and placement they were tested for, this demonstrates that these asset placement functions have the capacity to generate dungeons populated with assets without blocking essential pathways.

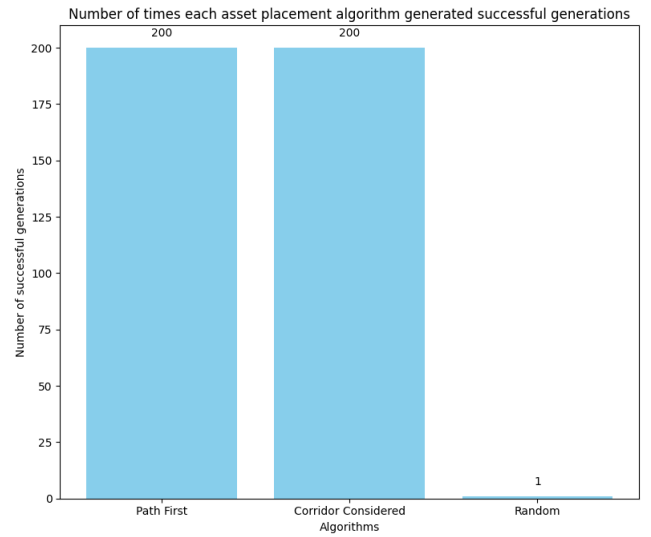


Fig. 6. Screenshot of bar chart showing how each of the asset placement algorithms performed

For further evidence comparing the algorithms against each other, this table contains the results of the post-hoc fisher exact tests that compared each algorithm against each other.

Compared algorithms	Path first	Corridor considered	Random
Path first	N/A	1.0	3.905^{-117}
Corridor considered	1.0	N/A	3.905^{-117}
Random	3.905^{-117}	3.905^{-117}	N/A

Table showing the p-values of the post-hoc fisher exact tests done comparing each algorithm

As seen in this table the path first and corridor considered algorithms' p-value is 1.0, this means that there is no difference between them. However, when random is compared to the other two algorithms there is a p-value of 3.905^{-117} this value is extremely small and shows that there is a huge difference in how the random algorithm performs compared to the path first and corridor considered placement methods.

After analyzing the data it is clear that the path first and corridor considered placement methods can be suitable methods for placing assets throughout a dungeon however the random method is not suitable, also the alternative hypothesis about the path first placement method performing the best was refuted as the p values from the fisher exact tests show that the corridor considered and path first asset placement methods performed the same.

XI. DISCUSSION

The gathered data supported the research question that asset placement algorithms can be used to populate a dungeon with assets without blocking the path as the corridor considered and path first placement methods generated valid iterations each time they were ran.

However, the artifact currently relies too heavily on hard-coded aspects such as getting the tiles that are made in the level generation section, for projects that generate their levels differently it may be difficult to adapt the artifact to suit that purpose. For this reason, the author suspects that indie developers and smaller studios would not be able to currently use the artifact to help with their development, which was one of the original aims when working on the artifact and on this paper.

Although the asset placement algorithm's results prove that they could work in a video game's context, they are currently placed almost randomly throughout valid areas which may work for destructible assets however for non-destructible ones this would make getting around the rooms difficult and hamper gameplay if these asset placement techniques were used in a game like 'Enter The Gungeon' [28] for example. To help solve this problem the placement algorithms could be improved, analyzing more details about the room size and where the walls are located. This could be useful for placing assets in more specific and appropriate places, such as placing banners on top wall spaces, crates or barrels against side and bottom walls, and larger assets such as tables where there is space for them. These changes would aid player immersion and also lend themselves to gameplay by ensuring the assets are not blocking the player from exploring each room fully.

To make the artifact easier to use it could be modified to adapt to other level generation techniques. For example, instead of using tile sets, the floor could be made out of a series of game objects that contain the floor tag, the placement algorithms could search for objects with this tag and analyze their size and positions to determine where to place the assets. This would be a more suitable and simple way for other projects made in Unity [9] to do automatic asset placement.

XII. CONCLUSION

While the computing artifact in its current state is likely not suitable to be used for other projects like the author was hoping it would be, it still served its purpose for this paper, the results of the research question and hypothesis highlights a possibility for automatic asset placement to be viable in game development, especially for solo or small team indie developers.

As a next step that could be taken the computing artifact could be modified to make it more adaptable and modular to support other level generation techniques as discussed in the discussion section. These changes could change the current artifact from being an admittedly simple one that places assets almost completely randomly, only ensuring that there is a valid path in the case of the path first and corridor considered algorithms, to a powerful tool that would aid a solo, or small team of developer's game development projects, saving time and creating immersive environments that would encourage replayability.

In conclusion, the author believes that this paper demonstrates the potential of automated asset placement algorithms as a potential tool for game development. While there are

improvements and modifications to be made to the artifact made for this paper specifically, the success of the path first and corridor considered asset placement methods provides a strong foundation for possible future investigation. With further work and polish, tools utilizing automatic asset placement algorithms could help game developers by reducing development time that would be used by placing assets manually and helping smaller teams or solo developers create more immersive and re-playable environments.

Github Repository link: <https://github.falmouth.ac.uk/GA-Undergrad-Student-Work-24-25/COMP302-Artifact-Unity>

Ethics application link: <https://learningspace.falmouth.ac.uk/mod/forum/discuss.php?d=56639>

REFERENCES

- [1] Kenny. 'Procedural Generation: An Overview'. *Medium*, 24 Feb. 2021, <https://kentpawson123.medium.com/procedural-generation-an-overview-1b054a0f8d41>.
- [2] *Rogue*. <https://www.pcjs.org/software/pcx86/game/other/1985/rogue/> Developed 1980, Made by Michael Toy and Glenn Wichman with later contributions by Ken Arnold.
- [3] 'Procedural Generation'. Wikipedia, 6 Sept. 2024. Wikipedia, https://en.wikipedia.org/w/index.php?title=Procedural_generation&oldid=1244382520.
- [4] Thompson, Tommy. Scalable Level Generation for 2D Platforming Games. Aug. 2023. [aru.figshare.com, https://aru.figshare.com/articles/conference_contribution/Scalable_Level_Generation_for_2D_Platforming_Games/23757027/1?file=42194556](https://aru.figshare.com/articles/conference_contribution/Scalable_Level_Generation_for_2D_Platforming_Games/23757027/1?file=42194556)
- [5] Sunny Valley Studio - Unity Procedural Dungeon Generation'. YouTube, <http://www.youtube.com/playlist?list=PLcRSafycjWFenI87z7uZHFv6cUG2Tzu9v>. Published Dec. 2020
- [6] 'Random Walk (Implementation in Python)'. Geeks-forGeeks, 22 Oct. 2017, <https://www.geeksforgeeks.org/random-walk-implementation-python/>.
- [7] 'Binary Space Partitioning'. GeeksforGeeks, 26 June 2020, <https://www.geeksforgeeks.org/binary-space-partitioning/>.
- [8] G*Power. <https://www.psychologie.hhu.de/arbeitsgruppen/allgemeine-psychologie-und-arbeitspsychologie/gpower>. Accessed 11 Apr. 2025.
- [9] "Unity Real-Time Development Platform — 3D, 2D, VR AR Engine." Unity, <https://unity.com>. Accessed 3 Dec. 2024.
- [10] Shaffer, Timothy Dresden, et al. Replayability of Video Games. 17 May 2011. https://digital.wpi.edu/concern/student_works/g158bh73h?locale=en
- [11] J. Togelius, G. N. Yannakakis, K. O. Stanley and C. Browne, "Search-Based Procedural Content Generation: A Taxonomy and Survey," in IEEE Transactions on Computational Intelligence and AI in Games, vol. 3, no. 3, pp. 172-186, Sept. 2011, <https://doi.org/10.1109/TCIAIG.2011.2148116> keywords: Games; Algorithm design and analysis; Optimization; Evolutionary computation; Encoding; Weapons; Buildings; Computer graphics; design automation; evolutionary computation; genetic algorithms,
- [12] Cui, Xiao Shi, Hao. (2010). A*-based Pathfinding in Modern Computer Games. 11. https://www.researchgate.net/publication/267809499_A-based_Pathfinding_in_Modern_Computer_Games
- [13] TaroDev Pathfinding - Understanding A* (A star) [Online Video]. Available: <https://www.youtube.com/watch?v=i0x5fj4PqP4>. Accessed 1 Dec. 2024. Published 16 Nov 2021.
- [14] Sure Footing on Steam. https://store.steampowered.com/app/670730/Sure_Footing/. Accessed 11 Apr. 2025.
- [15] Depth-First Search (DFS) — Brilliant Math Science Wiki. <https://brilliant.org/wiki/depth-first-search-dfs/>. Accessed 2 Dec. 2024.
- [16] Beukman, Michael, et al. "Hierarchical WaveFunction Collapse." Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, vol. 19, no. 1, Oct. 2023, pp. 23–33. ojs.aaai.org, <https://doi.org/10.1609/aiide.v19i1.27498>.
- [17] Mao, Xinyu, et al. Procedural Content Generation via Generative Artificial Intelligence. arXiv:2407.09013, arXiv, 12 July 2024. [arXiv.org, https://doi.org/10.48550/arXiv.2407.09013](https://doi.org/10.48550/arXiv.2407.09013).

- [18] M. Beukman et al., "Hierarchically Composing Level Generators for the Creation of Complex Structures," in IEEE Transactions on Games, vol. 16, no. 2, pp. 459-469, June 2024, doi: 10.1109/TG.2023.3297619. keywords: Games;Generators;Task analysis;Genetic algorithms;Statistics;Sociology;Search problems;Evolutionary computation;genetic algorithms;procedural content generation. Available: <https://ieeexplore.ieee.org/document/10190159>
- [19] "2D Pixel Dungeon Asset Pack by Pixel_Poem." Itch.io, <https://pixel-poem.itch.io/dungeon-assetpack>. Accessed 11 Apr. 2025.
- [20] Rapid Application Development (RAD) — Definition, Steps & Full Guide. <https://kissflow.com/application-development/rad/rapid-application-development/>. Accessed 3 Dec. 2024.
- [21] Deming, W. Edwards. 'Improvement of Quality and Productivity through Action by Management'. National Productivity Review, vol. 1, no. 1, Dec. 1981, pp. 12–22. DOI.org (Crossref), <https://doi.org/10.1002/npr.4040010105>.
- [22] 'Differences between Functional and Non-Functional Testing'. GeeksforGeeks, 8 May 2019, <https://www.geeksforgeeks.org/differences-between-functional-and-non-functional-testing/>.
- [23] Ajay. 'Introduction to Epistemology, Ontology and Axiology in Philosophy'. Medium, 23 Aug. 2021, https://medium.com/@softdev_ajay/introduction-to-epistemology-ontology-and-axiology-in-philosophy-5a2fc0b726ac.
- [24] Contributors, NU Editorial. 'What Is Qualitative vs. Quantitative Study?' National University, 27 Apr. 2023, <https://www.nu.edu/blog/qualitative-vs-quantitative-study/>.
- [25] Half-Life 2 on Steam. https://store.steampowered.com/app/220/HalfLife_2/. Released 2004 Made by Valve
- [26] Statorials, YouTube. <https://www.youtube.com/watch?v=tM5v-GMs0Qs>. Accessed 11 Apr. 2025.
- [27] "The Fisher's Exact Test." Technology Networks, <http://www.technologynetworks.com/tn/articles/the-fishers-exact-test-385738>. Accessed 11 Apr. 2025.
- [28] Enter the Gungeon on Steam. https://store.steampowered.com/app/311690/Enter_the_Gungeon/. Accessed 11 Apr. 2025.
- [29] Capture, Organize, and Tackle Your to-Dos from Anywhere — Trello. <https://trello.com/>. Accessed 11 Apr. 2025.

XIII. APPENDIX

A. Acknowledgments

The author would like to thank Sokol Murturi (Supervisor), Joseph Walton Rivers (Supervisor), and Michael Scott (Course Leader) for their guidance and support during the creation of this paper and computing artifact as well as my peers for their support and encouragement. The author would also like to thank Peter from Sunny Valley Studios [5] for help with the level generation aspect of the artifact and TaroDev [13] for help with the A* pathfinding part of the artifact as well as Pixel Poem [19] for the tile set that was used to represent the dungeon used in the artifact.

B. Data gathering and analysis - testing details

Quality assurance was previously discussed, however, to summarize, python was used to run a chi-square test of independence to determine if there was a difference in how the algorithms performed. This test demonstrated that there was a massive difference between how at least one of the algorithms performed compared to at least one other. In order to narrow this down post-hoc testing was done using fisher-exact tests which determined that the random algorithm was the one that was being heavily outperformed by the other two algorithms. The code for these tests and also the code that generated the bar chart in Figure 6 can be seen in Figure 4 and also in the results folder of the GitHub repository. Videos of the author testing each asset placement algorithm when collecting data can also be seen in the results folder of the GitHub repository.

Listing 4. Code used for data analysis as well as for generating bar chart showing asset placement algorithm results

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as pyplot
from scipy.stats import fisher_exact

# Format: [passes, fails]
results = {
    "PathFirst": [200, 0],
    "CorridorConsidered": [200, 0],
    "Random": [1, 199]
}

# Create a contingency table for the
# chi-square test
observed = np.array([
    results["PathFirst"],
    results["CorridorConsidered"],
    results["Random"]
])

# Run Chi-Square Test of Independence
chi2, p, dof, expected =
    stats.chi2_contingency(observed)

# Print results
print("Chi-Square_test_results:")
print(f"Chi2_Statistic:_{chi2:.4f}")
print(f"P-value:_{p:.4f}")

pathFirstVsRandom = np.array([
    results["PathFirst"],
    results["Random"]
])
oddsratio, p_value =
    fisher_exact(pathFirstVsRandom)
print("PathFirst_vs_Random_p_value:", p_value)

corridorConsideredVsRandom = np.array([
    results["CorridorConsidered"],
    results["Random"]
])
oddsratio, p_value =
    fisher_exact(corridorConsideredVsRandom)
print("CorridorConsidered_vs_Random_p_
    value:", p_value)

corridorConsideredVsPathFirst = np.array([
    results["CorridorConsidered"],
    results["PathFirst"]
])
oddsratio, p_value =
    fisher_exact(corridorConsideredVsPathFirst)
print("CorridorConsidered_vs_PathFirst_p_
    value:", p_value)

#Generating bar chart of results
algorithmsToPlot = ["Path_First", "Corridor_
    Considered", "Random"]
resultsToPlot = [200, 200, 1]

pyplot.bar(algorithmsToPlot, resultsToPlot,
    color="skyblue")
pyplot.xlabel("Algorithms")
```

```

pyplot.ylabel("Number_of_successful_
    ↳ generations")
pyplot.title("Number_of_times_each_asset_
    ↳ placement_algorithm_generated_
    ↳ successful_generations")

for i, value in enumerate(resultsToPlot):
    pyplot.text(i, value + 5, str(value),
        ↳ ha='center')

pyplot.show()

```

C. Quality assurance - Unit testing details

To ensure the quality of the computing artifact unit tests were made and ran. These tests being ran can be seen in Figures 7 and 8. The edit mode tests cover making sure the direction values which are used in level generation and getting the neighbors of each tile are all correct, as well as ensuring the binary space partitioning and random walk algorithm's code is working as expected.

The play mode tests cover room first generation which is the method that is used by all the asset placement functions as well as ensuring the functionality of the base code that all the asset placement functions use, these being making sure that assets do not spawn on door tiles as this causes the pathfinding code to break, as well as ensuring the assets spawn correctly, the random variables are set to ensure that the assets would spawn if the code was working correctly. In the "RandomSpawnNotSpawningWhenGuranteedToFail_Test" the random variables are set to ensure that an asset will not be spawned if the code is functioning properly and the test checks that this is the case. The code for the tests can be found in the tests folder of the computing artifact, available on the GitHub repository.

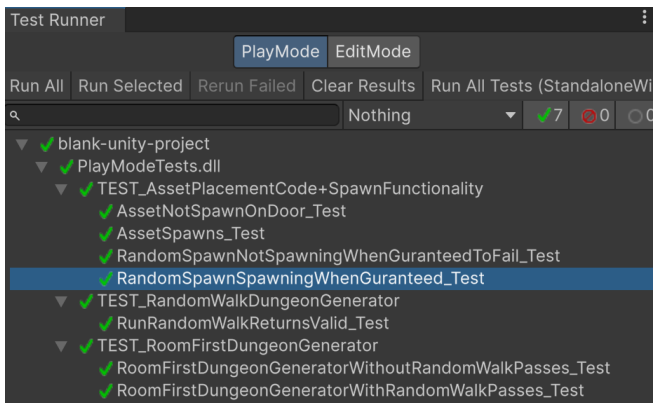


Fig. 7. Screenshot of play mode unit tests

D. Critical addendum - self reflection

I, the author am proud of how much work I feel I have put into this project however I am frustrated that I had to rely on tutorials for features like level generation and pathfinding, while I realize that my paper is focused on the asset placement functions and analyzing them I would have liked if I could

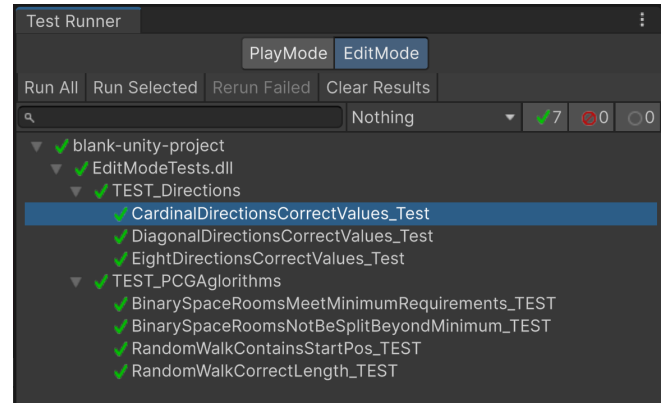


Fig. 8. Screenshot of edit mode unit tests

have written all the code myself. However, I believe this would have been too much work for me to undertake at my current skill level and would have caused the artifact and paper to suffer in terms of quality.

I believe one of the main things that helped me was learning from other developers, for example when looking through the tutorial for dungeon generation by Peter from Sunny Valley Studio [5] I not only learned about how to do level generation and learned about the binary space partitioning and random walk algorithms, I also learned about small things that I have been able to apply to my other programming projects.

Some examples of these are that you can highlight a variable, hold control, and double tap R to rename a variable across a whole solution, you can make a new method quickly by first typing out the method name and filling out the parameters where you want it to be called and then right clicking and generating the method which then creates the method using the parameters that have been assigned to it. I also learned that you can highlight code and extract it into a method, this helped refactor other code in different projects. In the future, I am going to strive to pay more attention to practices that other developers employ so I can incorporate them into my workflow and become a more efficient and experienced programmer as I go into the working world.

I believe my engagement with this project could have been better, towards the start of the project I focused more on my other assignments as I placed them at a higher priority as they were due first, I also find myself feeling bogged down when there are multiple projects to work on at the same as I tend to work on one at a time until it is complete and then move on to the next. While I do not believe that focusing on the projects due earlier was the incorrect decision I do regret that I did not spread out my time more evenly, as this project's due date came up it also caused me to, unfortunately, neglect my group project which has not been fair on my teammates. I intend to remedy this after I hand in this project. To improve this situation in the future I plan to timetable out my time more using a calendar management app, splitting up my time based on priority but not completely removing a task that is of lower priority. I believe this will keep me organized and help

me not fall behind on tasks that are due further in the future because I leave them until the last moment.

Towards the end of the project I had not gotten as far as I had hoped, it took me a lot longer than my peers to finish my computing artifact, partly due to me putting off the task, partly due to other forces outside my control which resulted in me finding myself very behind, this made me feel overwhelmed and frustrated when it came to writing up the paper, to combat this I received advice from a peer who told me to lay out and clearly define my tasks in a checklist. I used this advice and laid out what I needed to write and do for each section as well as any changes I had to make to my computing artifact, this helped immensely as instead of thinking about what I had to do and assigning priority in the moment I had a list that was already laid out and I could take tasks from, it also ensured I was not forgetting anything. This is something that I will use in the future when dealing with large tasks as I believe it makes them more manageable and less daunting.

I believe the RAD cycle of development was a strong choice for the computing artifact, it allowed me to get the artifact together quickly once I started focusing my efforts on finishing it. However, it also caused some issues, one of these being that as I had not properly planned out the development of the artifact I realized rather late into the project that I had to re-scope, if I had used a different cycle of development that involved more planning I likely could have re-scoped earlier and therefore had more time to polish my artifact and work on the paper. However I still believe the RAD cycle of development fits my workflow, to adapt it to suit my future projects better I am going to combine it with Trello [29], a task management tool, which I aim to use as a checklist, allowing me to break large programming projects into small tasks, as mentioned previously I believe this helps keep me organized while not encroaching too much on time that RAD saves by not having a focus on planning that I believe is one of its main benefits.

I also learned a valuable lesson about over-scoping and rushing into things without first properly thinking them through, I believe I over-scoped in the beginning and did not put enough thought into my original hypothesis and its practical viability, my original hypothesis was comparing how online and offline generation compared for placing assets throughout generated dungeon levels, however, I had to change my hypothesis to the current one late in development as I realized it would be a lot of work to develop two rather different methods of level generation and asset placement as well as record meaningful results for the differences between the two. I believe that the over-scoping I did for this project harmed the quality of the artifact rather significantly, if I had my new hypothesis from the beginning I could have created more asset placement algorithms such as one that examines the space to a more sophisticated level thus resulting in more interesting results and information.

In the future, I am going to strive to think more about the logistics of projects I want to undertake as well as my current skill level to make more educated decisions on whether they

are viable and would have a purpose. I would also like to adapt the computing artifact into a 3D dungeon generator and asset placement system that places assets in more appropriate places for a solo virtual reality dungeon crawler project I would like to work on, I want to apply the lessons I have discussed in this section to make sure this project's code base is cleaner and more efficient than that of previous projects I have worked on.